

Laporan Praktikum Kontrol Cerdas

Minggu ke - 4

Nama : Mahesswara Aryasatya Nugraha
NIM : 224308036
Kelas : TKA-6B
Akun Github : <https://github.com/maheswaraaryasatya>
Student Lab Assistant : Mas Alfian

1. Judul Percobaan : ***Reinforcement Learning for Autonomous Control with DQN***

2. Tujuan Percobaan :

- Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- Mengimplemtasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
- Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunaka GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori :

Reinforcement Learning adalah algoritma yang mengumpulkan pengetahuan (sinyal penguatan) untuk memilih tindakan yang mengarah ke hasil tertinggi yang diharapkan. Kemampuan *reinforcement learning* untuk secara mandiri menjelajahi lingkungan yang sangat dinamis dan stokastik dan mengembangkan, dengan mengumpulkan umpan balik evaluatif dari lingkungan, kebijakan kontrol yang optimal (**Karimah, 2023**). Tidak seperti *machine learning* yang membutuhkan dataset untuk proses training, pada *reinforcement learning* agen akan mengeksplorasi lingkungannya dan membuat keputusan berdasarkan nilai reward dan punishment yang diberikan ketika agen melakukan suatu aksi (**Kurniawati dkk., 2021**). Setelah proses trial dan error, agen akan mempelajari aksi apa yang harus dilakukan untuk mendapatkan reward dengan nilai tertinggi. Sehingga agen akan memiliki kecenderungan

untuk mengambil aksi tersebut jika berada pada suatu kondisi tertentu. *Epsilon greedy* merupakan salah satu algoritma yang digunakan untuk *learning process*. Algoritma ini menyeimbangkan proses eksploitasi dan eksplorasi berdasarkan nilai epsilon. Ketika agen melakukan eksplorasi lingkungan, agen akan mengambil aksi acak tanpa mempedulikan nilai reward atau punishment yang didapatkan, sedangkan ketika agen berada dalam mode eksploitasi, maka agen akan melakukan aksi yang memberikan nilai reward tertinggi (Liu dkk., 2022). Epsilon dalam hal ini merupakan parameter yang menentukan apakah agen akan melakukan eksplorasi atau eksploitasi. Pada praktikum ini dengan mengimplemtasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN). DQN adalah algoritma RL yang menggabungkan algoritma *Q-Learning* dengan *Deep Neural Network* (Putra dkk., 2024). *Q-Learning* dikategorikan dalam metode model-free yang tidak membutuhkan informasi yang lengkap tentang kondisi *environment*. Berbeda dengan metode *model-based*, *Q-Learning* memungkinkan agen untuk berada satu langkah ke depan. Agen berada dalam keadaan tertentu, mengambil tindakan tertentu, dan menerima value berdasarkan tindakan tersebut. Pada DQN, DNN digunakan untuk memodelkan nilai value tersebut dengan menggunakan *Neural Network*.

4. Analisis dan Diskusi :

-Analisis

MountainCar-v0 adalah lingkungan di mana sebuah mobil harus mencapai puncak bukit meskipun tidak memiliki daya yang cukup untuk langsung mencapainya. Lingkungan ini memiliki ruang keadaan dua dimensi, yang terdiri dari posisi mobil di sepanjang bukit dan kecepatan mobil. Terdapat tiga aksi yang dapat diambil oleh agen, yaitu bergerak ke kiri, tetap diam, atau bergerak ke kanan. Agen harus memilih tindakan yang tepat untuk mengoptimalkan pergerakan mobil. Dalam lingkungan ini, agen menerima reward standar sebesar -1 untuk setiap langkah yang diambil, yang mendorong agen untuk mencapai puncak bukit secepat mungkin.

Namun, dalam beberapa implementasi, reward ini dapat dimodifikasi untuk mempercepat proses pembelajaran. Tantangan utama adalah bahwa mobil harus mengayun bolak-balik untuk mendapatkan momentum yang cukup agar dapat mencapai puncak bukit. Reinforcement Learning (RL) digunakan untuk melatih agen dalam lingkungan ini. Agen belajar melalui interaksi dengan lingkungan, mencoba berbagai tindakan, dan menerima umpan balik dalam bentuk reward atau punishment. Dengan menggunakan algoritma RL, agen dapat mengembangkan kebijakan yang optimal untuk mencapai tujuan, belajar dari pengalaman sebelumnya, dan beradaptasi dengan kondisi yang berubah. Karena sifatnya yang kontinu. Penggunaan Deep Q-Network (DQN) menjadi pendekatan yang baik, karena DQN dapat menangani ruang keadaan yang besar dan kompleks dengan menggunakan jaringan saraf untuk memperkirakan nilai Q, sehingga memungkinkan agen untuk belajar dengan lebih efisien dalam mencapai puncak bukit.

-Diskusi

Deep Q-Network (DQN) pada lingkungan MountainCar-v0 bertujuan untuk melatih agen agar dapat mencapai puncak bukit dengan menggunakan aksi yang optimal. Lingkungan ini menantang karena mobil harus mengumpulkan momentum dengan cara mengayun bolak-balik sebelum dapat mencapai tujuan. Model DQN yang digunakan memiliki struktur sederhana dengan tiga lapisan. Lapisan input terdiri dari 24 neuron dengan aktivasi ReLU, lapisan hidden juga memiliki 24 neuron dengan aktivasi ReLU, dan lapisan output terdiri dari 3 neuron yang mewakili jumlah aksi dengan aktivasi linear. Tantangan utama yang dihadapi dalam implementasi DQN ini adalah waktu yang diperlukan oleh agen untuk menemukan strategi optimal. Pada fase awal pelatihan, agen sering kali melakukan aksi acak, yang dapat memperlambat proses pembelajaran.

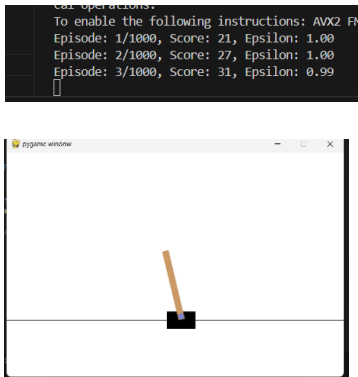
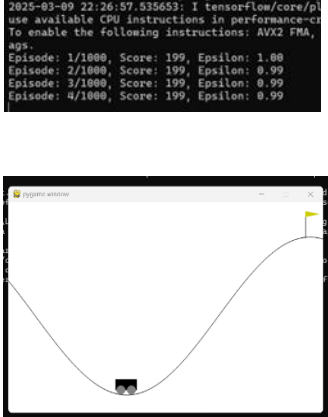
5. Assignment :

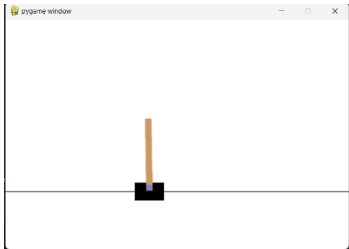
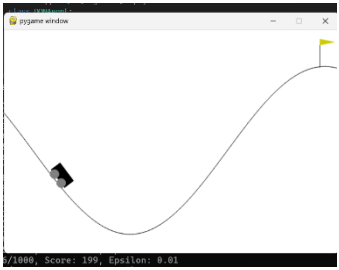
Tugas percobaan praktikum pada minggu keempat ini yaitu implementasi dari algoritma Deep Q-Network (DQN) yang digunakan untuk melatih environment agen dalam environment MountainCar-v0. Tujuan dari agen ini yaitu mencapai

puncak bukit dengan menggunakan aksi yang optimal. Kode dimulai dengan mengimpor beberapa library yang diperlukan, seperti `gymnasium` untuk `environment reinforcement learning`, `numpy` untuk operasi numerik, `random` untuk pemilihan acak, dan `deque` dari `collections` untuk menyimpan pengalaman. Selain itu, `tensorflow.keras` digunakan untuk membangun model jaringan saraf. Selanjutnya, terdapat kelas yang disebut `DQNAgent`, yang mendefinisikan agen DQN. Agen diinisialisasi dengan ukuran keadaan dan ukuran aksi. Beberapa parameter penting juga digunakan, seperti `gamma` yang berfungsi sebagai faktor diskon, `epsilon` yang merupakan parameter untuk eksplorasi, dan `learning_rate` yang digunakan untuk optimasi model.

Pada eksperimen pertama **CartPole-V1**, agen harus menjaga tiang tetap tegak di atas gerobak/kotak. Lingkungan ini cukup sederhana, sehingga agen dapat belajar dengan cepat. Setiap kali agen berhasil menjaga tiang tetap tegak, ia mendapatkan reward positif. Pelatihan untuk `CartPole` bisa dilakukan dalam waktu singkat, hanya dalam beberapa ratus episode. Agen dapat dengan stabil mencapai tujuannya karena lingkungan ini lebih mudah diprediksi. Sedangkan, pada **MountainCar-v0**, agen harus menggerakkan mobil ke puncak bukit, tetapi mobil tidak memiliki cukup tenaga untuk langsung mencapai tujuan. Agen harus belajar membangun momentum dengan bergerak maju dan mundur. Reward di **MountainCar-v0** lebih susah karena mobil tidak berhasil mencapai puncak, yang membuat pembelajaran menjadi lebih sulit. Model yang digunakan memerlukan arsitektur yang lebih kompleks atau teknik tambahan untuk meningkatkan stabilitas. Proses pelatihan untuk `MountainCar` memerlukan lebih banyak episode. sebelum agen dapat mencapai kinerja yang baik. Pembaruan nilai dalam lingkungan ini bisa lebih berfluktuasi, sehingga kembali ke skor yang lebih tinggi.

6. Data dan Output Hasil :

No.	Variabel	Hasil Pengamatan	
		Chart pole v1	Mountain Car v0
1.	Waktu Pelatihan	Chartpole lebih cepat dalam mencapai performa optimal saat episode diatas 100	Mountain car membutuhkan waktu lama untuk mencapai performa yang optimal
2.	Kinerja Agen	Kinerja yang optimal dengan skor yang lebih tinggi	Masih menghadapi tantangan untuk mencapai puncak
3.	Kondisi Berhasil	Episode berakhir jika tiang jatuh atau batas langkah maksimum tercapai	Episode berakhir jika mobil mencapai puncak bukit (flag)
4.	Hasil Episode Awal		

5.	Hasil Episode Akhir	<pre> Episode: 990/1000, Score: 176, Epsilon: 0.01 Episode: 991/1000, Score: 92, Epsilon: 0.01 Episode: 992/1000, Score: 200, Epsilon: 0.01 Episode: 993/1000, Score: 250, Epsilon: 0.01 Episode: 994/1000, Score: 192, Epsilon: 0.01 Episode: 995/1000, Score: 229, Epsilon: 0.01 Episode: 996/1000, Score: 201, Epsilon: 0.01 Episode: 997/1000, Score: 403, Epsilon: 0.01 Episode: 998/1000, Score: 176, Epsilon: 0.01 </pre> 	<pre> Episode: 991/1000, Score: 199, Epsilon: 0.01 Episode: 992/1000, Score: 199, Epsilon: 0.01 Episode: 993/1000, Score: 199, Epsilon: 0.01 Episode: 994/1000, Score: 199, Epsilon: 0.01 Episode: 995/1000, Score: 199, Epsilon: 0.01 Episode: 996/1000, Score: 199, Epsilon: 0.01 Episode: 997/1000, Score: 199, Epsilon: 0.01 Episode: 998/1000, Score: 199, Epsilon: 0.01 Episode: 999/1000, Score: 199, Epsilon: 0.01 </pre> 
----	---------------------	--	---

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- CartPole-v1 lebih cepat dipelajari karena memberikan *reward* di setiap langkah, sehingga agen dapat dengan cepat memahami strategi optimal.
- MountainCar-v0 lebih sulit dipelajari karena *reward* hanya diberikan saat mencapai puncak bukit, memerlukan strategi eksplorasi lebih lama, dan mengandalkan momentum untuk berhasil.
- Penggunaan target *network* membantu stabilitas pembelajaran, menghindari perubahan nilai Q yang terlalu drastis.
- *Experience Replay* meningkatkan efisiensi pembelajaran, tetapi ukuran buffer harus disesuaikan dengan kompleksitas lingkungan agar tetap efektif.
- *Hyperparameter* sangat menentukan keberhasilan agen, sehingga perlu disesuaikan dengan karakteristik lingkungan untuk hasil optimal.

8. Saran :

Untuk meningkatkan performa agen DQN pada lingkungan yang lebih kompleks, disarankan untuk menyesuaikan parameter lebih lanjut melalui eksperimen, seperti menggunakan *dueling DQN*, *prioritized experience replay*,

atau peningkatan arsitektur jaringan dengan lebih banyak neuron atau lapisan. Selain itu, untuk kasus seperti MountainCar, pendekatan lain seperti *actor-critic methods* atau *policy-based reinforcement learning* dapat dicoba untuk meningkatkan efisiensi pembelajaran. Lebih jauh, penggunaan *hardware* yang lebih kuat seperti GPU dapat mempercepat proses pelatihan, terutama pada lingkungan dengan kompleksitas tinggi.

9. Daftar Pustaka :

- Karimah Tauhid, Volume 2 Nomor 1 (2023), e-ISSN 2963-590X. (2023). 2.
- Kurniawati, N., Ningsih, Y. K., Puspa, S. D., & Adi, T. S. (2021). Algoritma Epsilon Greedy pada Reinforcement Learning untuk Modulasi Adaptif Komunikasi Vehicle to Infrastructure (V2I). ELKOMIKA: Jurnal Teknik Energi Elektrik, Teknik Telekomunikasi, & Teknik Elektronika, 9(3), 716. <https://doi.org/10.26760/elkomika.v9i3.716>
- Liu, F., Viano, L., & Cevher, V. (t.t.). Understanding Deep Neural Function Approximation in Reinforcement Learning via ϵ -Greedy Exploration.
- Putra, R. A., Syahbana, Y. A., & Ananda. (2024). Implementasi Algoritma Deep Q-Network (DQN) pada Lampu Lalu Lintas Adaptif Berdasarkan Waktu Tunggu dan Arus Kendaraan. The Indonesian Journal of Computer Science, 13(5). <https://doi.org/10.33022/ijcs.v13i5.4372>